❺ Finally, you can send the email.

This code is turning into something gnarly. Improving the design of `EmailSender` to separate out the responsibilities has made calling it from `RegisterUser` a real chore. This code has several problems:

- *Not obeying the SRP*—Now our code is responsible for both creating an `EmailSender` object and using it to send an email.
- *Considerable ceremony*—*Ceremony* refers to code that you have to write but that isn't adding value directly. Of the 11 lines of code in the `RegisterUser` method, only the last two are doing anything useful, which makes it harder to read and harder to understand the intent of the methods.
- *Tied to the implementation*—If you decide to refactor `EmailSender` and add another dependency, you'd need to update every place it's used. Likewise, if any dependencies are refactored, you would need to update this code too.
- *Hard to reuse instance*—In the example code we created new instances of all the objects. But what if creating a new `NetworkClient` is computationally expensive and we'd like to reuse instances? We'd have to add extra code to handle that task, further increasing the amount of boilerplate code.

`RegisterUser` has an *implicit* dependency on the `EmailSender` class, as it creates the object manually itself. The only way to know that `RegisterUser` uses `EmailSender` is to look at its source code. By contrast, `EmailSender` has *explicit* dependencies on `NetworkClient` and `MessageFactory`, which must be provided